

Here is the complete manual provided directly as a raw Markdown block so you can copy it easily or convert it to a PDF:

Django Lab Task: Creating a Post Model & Seeding Data with Faker

Objective

In this lab task, you will create a Django model named `Post` with 5 specific fields, regi

Prerequisites

Before you start, make sure you have:

- * A working Django project and a Django app already created and registered in `INSTALLED\_APPS`
- * Python installed on your computer.
- * Your terminal or command prompt open.

Part 1: Defining the `Post` Model

Models in Django represent the tables in your database. You will create a `Post` model ins

Step 1: Open `models.py`

Navigate to your app directory and open the `models.py` file.

Step 2: Write the Model Code

Add the following `Post` model. We will include 5 fields:

1. `title`: The headline of the post (a short text).
2. `content`: The main body or story of the post (a long text).
3. `author`: The name of the person who wrote it (a short text).
4. `created\_at`: The exact date and time when the post was created (automatically filled).
5. `is\_published`: A checkbox to see if the post is live or a draft (Yes/No boolean).

Write or copy this code into your file:

```
```python
from django.db import models

class Post(models.Model):
 title = models.CharField(max_length=200)
 content = models.TextField()
 author = models.CharField(max_length=100)
 created_at = models.DateTimeField(auto_now_add=True)
 is_published = models.BooleanField(default=False)

 def __str__(self):
 return self.title
```
```

Code Explanation:

- `models.CharField` : Used for short text. `max_length` restricts how many characters can be typed.
 - `models.TextField` : Used for long text (like paragraphs of content).
 - `models.DateTimeField(auto_now_add=True)` : Automatically saves the date and time exactly when a new post is saved. You don't have to fill this manually!
 - `models.BooleanField(default=False)` : Creates a True/False checkbox field. It starts as unchecked (`False`) by default.
 - `def __str__(self):` : This special function tells Django to show the actual **title** of the post in the admin panel, instead of something confusing like `Post object (1)`.
-

Part 2: Running Database Migrations

Whenever you create or change a model, you must let your database know so that it can update its structure. This is a two-step process.

Step 1: Create the Migration Blueprint

Open your terminal, make sure you are in the folder containing `manage.py`, and run this command:

```
python manage.py makemigrations
```

- **What this does:** Django looks at your `models.py`, notices the new `Post` model, and creates a plan file inside the `migrations/` folder. It doesn't modify the database yet!

Step 2: Apply the Migrations

Now, apply that plan to your actual database file by running:

```
python manage.py migrate
```

- **What this does:** Django reads the plan file and officially builds the `Post` table inside your database.
-

Part 3: Registering and Customizing Django Admin

Django provides a built-in administrative interface. We need to register our model so it shows up there, and customize it to be user-friendly.

Step 1: Open `admin.py`

Step 1: Open `admin.py`

Inside your app directory, open the `admin.py` file.

Step 2: Write the Admin Configuration

Modify the file to look exactly like this:

```
from django.contrib import admin
from .models import Post

@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    # Fields to display as columns in the admin list view
    list_display = ('title', 'author', 'is_published', 'created_at')

    # Sidebar filters to narrow down the posts easily
    list_filter = ('is_published', 'created_at')

    # Search bar functionality to search by title or author name
    search_fields = ('title', 'author')
```

Code Explanation:

- `@admin.register(Post)` : This tells Django that the configuration below belongs to our `Post` model.
- `list_display` : By default, Django only shows one column. Adding this tuple displays your data in a beautiful table format with columns for Title, Author, Published Status, and Creation Date.
- `list_filter` : This adds a filter box on the right side of the page. You can click options to filter posts by whether they are published, or filter them by date.
- `search_fields` : This adds a search bar at the top of the admin page. If you type a keyword, Django will scan through all post titles and author names to find matches.

Part 4: Seeding 200 Random Posts Using Faker

Typing 200 sample posts manually in the admin dashboard would take hours. Instead, we will write a quick automation script using the **Faker** package to populate our database with realistic fake data.

Step 1: Install the Faker Package

If you haven't installed `faker` yet, install it via your terminal:

```
pip install faker
```

Step 2: Create the Seed Script

In the root directory of your Django project (the same folder where `manage.py` lives), create a new, completely blank file named `seed_posts.py`.

Step 3: Write the Seeding Code

Copy and paste this script into `seed_posts.py`:

```
import os
import django
import random

# 1. Setup Django environment settings
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'myproject.settings') # <-- REPLACE 'mypr
django.setup()

# 2. Import your model and Faker
from myapp.models import Post # <-- REPLACE 'myapp' with your actual app name
from faker import Faker

# Initialize Faker
fake = Faker()

def generate_fake_posts(number_of_posts=200):
    print(f"Starting to inject {number_of_posts} random posts into the database...")

    for i in range(number_of_posts):
        # Generate fake data for each field
        fake_title = fake.sentence(nb_words=6) # Generates a short sentence
        fake_content = fake.paragraph(nb_sentences=5) # Generates a paragraph of text
        fake_author = fake.name() # Generates a full name
        fake_is_published = random.choice([True, False]) # Randomly picks True or False

        # Create and save the Post into the database
        Post.objects.create(
            title=fake_title,
            content=fake_content,
            author=fake_author,
            is_published=fake_is_published
        )

        # Simple progress logger
        if (i + 1) % 50 == 0:
            print(f"{i + 1} posts added successfully.")

    print("Data seeding completed! Check your Django Admin Panel.")

if __name__ == '__main__':
```

```
generate_fake_posts(200)
```

CRITICAL FIX before running: Look at lines 7 and 11.

- Replace `'myproject.settings'` with your real project name.
- Replace `from myapp.models` with your real app name.

Step 4: Run the Script

Go to your terminal and run the Python script:

```
python seed_posts.py
```

Watch the terminal display the success logs. Once it finishes, run your server (`python manage.py runserver`), navigate to your localhost admin portal (`/admin`), log in with a superuser account, and witness 200 fully populated, randomized posts with interactive search, filters, and display tables!

Submission Checklist

Before declaring the task done, verify you have accomplished the following:

1. ☐ Run `makemigrations` and `migrate` without errors.
 2. ☐ Created a superuser account (`python manage.py createsuperuser`) to log into `/admin` .
 3. ☐ Can see the **Posts** section in the admin panel.
 4. ☐ The admin displays columns for Title, Author, Published, and Date Created.
 5. ☐ The search bar and right-side filters work perfectly.
 6. ☐ Exactly 200 fake posts show up when you check the post table.
-